RBE201TC 20261013 Exercises

Download the Matlab codes file, extract it and open the required `.m` file. The package contains eight files, with `%%` separating examples and task requirements.

File	Subsections
`RBE201TC_20261013_2_1.m`	2.1.1
`RBE201TC_20261013_2_2.m`	2.2.1, 2.2.2, 2.2.3, 2.2.4
`RBE201TC_20261013_2_3.m`	2.3.1, 2.3.2
`RBE201TC_20261013_3_1.m`	3.1.1, 3.1.2, 3.1.3
`RBE201TC_20261013_3_2.m`	3.2.1, 3.2.2
`RBE201TC_20261013_3_3.m`	3.3.1, 3.3.2
`RBE201TC_20261013_3_4.m`	3.4.1, 3.4.2
`RBE201TC_20261013_Extension.m`	Extension topics

1. Run the complete baseline and inspect its output or figure. Read each task requirement and make the requested changes yourself, using no more than three added or edited lines. Rerun the entire baseline section and inspect the changes.
2. Only the first subsection in each file is enabled. Later baselines are block-commented with standalone `%{` and `%}` lines. Remove those two lines around the required baseline, place the cursor in its `%%` section and select **Run Section**. Task sections contain instructions as comments; edit the baseline to carry them out.
3. Restart from the original baseline for every task. Restore edits and comment unused baselines before switching sections.
4. Numerical comments describe the original inputs; use actual results after edits. Section **2.3.2** and **3-D Points, Arrows and Surfaces** remain optional.
5. `RBE201TC_20261013_3_4.m` includes `filterTemps` and `isEqualTol` as local functions at the end of the file. Keep the function support area intact and run callers in the same file.

2.1 Expressions, powers and roots

2.1.1 MATLAB Expressions, Powers and Roots

File: `RBE201TC_20261013_2_1.m`

Baseline example: run the whole section first.

```MATLAB
a = 2 + 3*4;        % 14
b = (2+3)*4;        % 20
c = 2^3;            % 8
e = -2^2;           % -4
f = (-2)^2;         % 4
sqrt([4 9 16])      % [2 3 4]
distance_m = sqrt(3^2+4^2); % 5
sin(pi/2)           % 1
sind(90)            % 1

% Display output.
disp([a b c e f distance_m]);
```

Task 1: Change Unary-minus Grouping

Start from the original baseline. Change only the definition of e so that it squares the whole negative number -2, keeping f unchanged. Rerun the section and compare e with f.

Task 2: Change a Distance Input

Start from the original baseline. In the definition of distance_m, change the displacement component from 3 m to 6 m while keeping the other component at 4 m. Rerun the section and observe the new distance in metres.

2.2 Vectors, matrices, shape and indexing

2.2.1 Rows, Columns and Dimensions

File: `RBE201TC_20261013_2_2.m`

Baseline example: run the whole section first.

```MATLAB
row = [10 20 30];   % 1-by-3
col = [10;20;30];   % 3-by-1
M = [1 2 3;4 5 6]; % 2-by-3
size(M)          % [2 3]
size(M,1)        % 2 rows
size(M,2)        % 3 columns
length(M)        % 3
numel(M)         % 6

% Display output.
disp(size(row));
disp(size(col));
```

Task 1: Change the Matrix to Two Rows and Four Columns

Start from the original baseline. Change only the definition of M to a two-row, four-column matrix: use 1, 2, 3, 4 in its first row and 5, 6, 7, 8 in its second row. Rerun the section and compare size, length and numel.

Task 2: Change a Column Vector into a Row Vector

Start from the original baseline. Change only the definition of col so that its existing values 10, 20, 30 form a row vector instead of a column vector. Keep row unchanged. Rerun the section and compare the two vector sizes.

2.2.2 Two-Dimensional Indexing and end

File: `RBE201TC_20261013_2_2.m`

Baseline example: run the whole section first.

```MATLAB
M = [1 2 3;4 5 6];
M(2,3)              % 6
M(2,:)              % [4 5 6]
M(:,2)              % [2;5]
M(:,2:3)            % [2 3;5 6]
M(end,:)            % [4 5 6]
M(1,end-1:end)      % [2 3]
M(end,1)            % 4
M(1,end)            % 3
```

Task 1: Add a Row and Observe end

Start from the original baseline. Extend the definition of M by adding a third row containing 7, 8, 9, keeping its existing two rows unchanged. Rerun the section and observe how end affects the last-row and last-row-first-column outputs, and how the whole second column changes.

Task 2: Display the First and Last Rows Together

Start from the original baseline. Replace the line that displays M's second row with two lines: store its first and last rows together in selectedRows, in that order, then display selectedRows. Use end for the last row and retain every column. Rerun the section and observe the selected rows' order and shape.

2.2.3 Linear Indexing and Transpose

File: `RBE201TC_20261013_2_2.m`

Baseline example: run the whole section first.

```MATLAB
M = [1 2 3;4 5 6];
M(:)  % [1;4;2;5;3;6]
M(3)  % 2
M(end) % 6
M.'   % [1 4;2 5;3 6]
z = [1+2i 3];
z.'   % [1+2i;3]
z'    % [1-2i;3]
```

Task 1: Display Two Indexing Methods Side by Side

Start from the original baseline. Replace the third-linear-element display with three lines: store that element in linearValue, store the element at row 1 and column 3 in rowColumnValue, then display the two values side by side. Rerun the section and compare the two indexing methods.

Task 2: Change the Complex Input to Real Values

Start from the original baseline. Change only z's definition by removing the imaginary component from its first value, keeping its real part and its second value unchanged. Rerun the section and compare the nonconjugating and conjugate transpose outputs.

2.2.4 Array Modification and Generation

File: `RBE201TC_20261013_2_2.m`

Baseline example: run the whole section first.

```MATLAB
M = [1 2 3;4 5 6];
M(2,1) = 40;
M(:,2) = [];
M   % [1 3;40 6]
0:0.25:1
% [0 0.25 0.5 0.75 1]
linspace(0,1,5)
% [0 0.25 0.5 0.75 1]
zeros(2,3); ones(2,3); eye(2);

% Display output.
disp(zeros(2,3));
disp(ones(2,3));
disp(eye(2));
```

Task 1: Change Column Deletion to Row Deletion

Start from the original baseline. Change only the deletion statement so that it deletes M's entire second row instead of its second column. Keep the preceding change of an element to 40. Rerun the section and observe M's remaining values and shape.

Task 2: Use Three linspace Points

Start from the original baseline. Replace the linspace display with two lines: generate three equally spaced points from 0 to 1 in points, then display points. Keep the preceding colon sequence unchanged. Rerun the section and compare the spacing, number of points and endpoints.

2.3 Array arithmetic and linear algebra

2.3.1 Elementwise and Matrix Arithmetic

File: `RBE201TC_20261013_2_3.m`

Baseline example: run the whole section first.

```MATLAB
A = [1 2;3 4];
B = [5 6;7 8];
A.*B
A*B
A.^2
A^2
[10 20 30]./[2 4 5]
```

Task 1: Change B to the Identity

Start from the original baseline. Change only B's definition to a two-by-two identity matrix using eye. Keep A unchanged. Rerun the section and compare the elementwise product with the matrix product.

Task 2: Display the Difference Between the Two Squares

Start from the original baseline. Add two lines at the end: store the matrix square of A minus its elementwise square in difference, then display difference. Rerun the section and observe the entries and shape of the difference.

2.3.2 Linear Systems and Vectors (Optional)

File: `RBE201TC_20261013_2_3.m`

Baseline example: run the whole section first.

```MATLAB
C = [2 3;4 1]; b = [5;11];
x = C\b;    % [2.8;-0.2]
C*x-b       % near [0;0]
A = [1 2;3 4];
det(A)      % the determinant of A is -2
inv(A)      % the inverse of A is [-2 1;1.5 -0.5]
u = [1 2 3]; v = [4 5 6];
dot(u,v)    % 32
w = cross(u,v) % [-3 6 -3]
dot(u,w)    % 0
dot(v,w)    % 0
sqrt([4 9])   % [2 3]
```

Task 1: Change the Right-hand Side

Optional. Start from the original baseline. In the line defining C and b, change only b to a column vector whose first value is 8 and second value is 10. Keep C unchanged. Rerun the section and observe the solution x and the displayed residual of the linear system.

Task 2: Reverse the Cross-product Order

Optional. Start from the original baseline. Add three lines at the end: store the cross product with v first and u second in reverseW, display reverseW, then display its sum with the original w. Rerun the section and observe how reversing the input order affects the vector and the sum.

3.1 Comparisons, logic and filtering

3.1.1 Comparisons and Logical Arrays

File: `RBE201TC_20261013_3_1.m`

Baseline example: run the whole section first.

```MATLAB
A = [2 4 6 8]; B = [1 4 10 8];
A >= B          % [1 1 0 1] logical
A == B          % [0 1 0 1] logical
A ~= B          % [1 0 1 0] logical
v = [10 20 30 40];
mask = (v>=20) & (v<40); % [0 1 1 0]
v(mask)         % [20 30]
find(mask)      % [2 3]
sum(mask)       % 2
```

Task 1: include 40

Add up to three lines to create maskClosed for values of v from 20 to 40, including both endpoints. Display the selected values, their original indices and their count. Run the whole section and compare the results with the original mask.

Task 2: compare OR with AND

Restart from the original baseline. Add up to three lines to create maskOr by joining the conditions 'at least 20' and 'below 40' with OR. Display the mask, selected values and count, then compare them with the original AND selection.

3.1.2 Batch Logic and Short-Circuit Logic

File: `RBE201TC_20261013_3_1.m`

Baseline example: run the whole section first.

```matlab
v = [10 20 30 40];
mask = (v>=20) & (v<40);
otherMask = [false true false true];
andMask = mask & otherMask;
orMask = mask | otherMask;
notMask = ~mask;
anyVector = any(mask);
allVector = all(mask);
left = true; right = false;
scalarAnd = left && right;
scalarOr = left || right;
M = [true false true; false false true];
anyColumns = any(M);
allColumns = all(M);
anyWhole = any(M(:));
allWhole = all(M(:));
readings=[]; high=~isempty(readings) && max(readings)>40;
noDataOrHigh = isempty(readings) || max(readings)>40;
disp(andMask); disp(orMask); disp(notMask);
disp([anyVector allVector scalarAnd scalarOr]);
disp(anyColumns); disp(allColumns);
disp([anyWhole allWhole high noDataOrHigh]);
```

Task 1: change the matrix's logical entries

Change the definition of M so its first row contains three true values and its second row contains true, false and true, in that order. Run the section and compare the values and dimensions of anyColumns, allColumns, anyWhole and allWhole.

Task 2: evaluate the guarded data condition

Restart from the original baseline. Change readings from an empty array to a row vector containing 39 and 45, while retaining the guarded condition for high. Run the section, compare high with noDataOrHigh, and trace whether each short-circuit operator evaluates its right-hand condition.

3.1.3 Filtered Values and Original Positions

File: `RBE201TC_20261013_3_1.m`

Baseline example: run the whole section first.

```MATLAB
T_C = [35.2 36.5 NaN 37.8 38.1 36.0];
keep = ~isnan(T_C) & (T_C>=36) & (T_C<=38);
T_good_C = T_C(keep);      % [36.5 37.8 36.0]
originalIndex = find(keep); % [2 4 6]
numberKept = sum(keep);    % 3
isnan(T_C)                 % [0 0 1 0 0 0]
isfinite([1 NaN Inf])      % [1 0 0]
disp(T_good_C); disp(originalIndex); disp(numberKept);
```

Task 1: check endpoints and nonfinite values

Add up to three lines to create a row vector named T_boundary containing 36, 38, NaN, Inf and -Inf, in that order. Create keep_boundary to retain only finite temperatures from 36 to 38 degrees Celsius, including both endpoints. Display the retained values and their original indices, then run the section.

Task 2: narrow the filtering interval

Restart from the original baseline. Add up to three lines to create keepNarrow for non-NaN values of T_C that are at least 36 but below 37 degrees Celsius. Display the retained values, original indices and count. Run the section and compare the new selection with the original interval.

3.2 Strings and branches

3.2.1 Strings: Full Equality and Substrings

File: `RBE201TC_20261013_3_2.m`

Baseline example: run the whole section first.

```MATLAB
commands = ["Start" "STOP" ...
        "Pause" "stop"];
strcmp(commands,"stop")
% [0 0 0 1]
strcmpi(commands,"stop")
% [0 1 0 1]
"stop" == "stop"   % true
files = ["report_Q1.pdf" ...
 "data_backup.mat" "Q1_results.csv"];
hasQ1 = contains(files,"Q1");
% [1 0 1]
files(hasQ1)
% report_Q1.pdf, Q1_results.csv
chars = 'stop';
charEqual = (chars=='stop');
disp(hasQ1); disp(charEqual);
```

Task 1: change target-command case

Add up to two lines to define a string target containing uppercase STOP and display the results of matching commands to this target with strcmp and strcmpi. Run the section and identify which command positions each matching rule selects.

Task 2: add case-insensitive substring search

Restart from the original baseline. Add up to three lines to search files for the lowercase substring q1 using contains. Store the default case-sensitive result in sensitiveQ1 and the result with IgnoreCase enabled in insensitiveQ1. Display the default mask and the filenames selected by the case-insensitive mask, then run the section.

3.2.2 Branching and Discrete States

File: `RBE201TC_20261013_3_2.m`

Baseline example: run the whole section first.

```MATLAB
mass_g = 144;
if mass_g <= 120
    category = "light";
elseif mass_g <= 160
    category = "medium";
else
    category = "heavy";
end
disp(category)  % medium
cmd = 'RUN';
switch lower(cmd)
    case 'run'
        disp('Start movement');
    case 'stop'
        disp('Stop movement');
    case 'pause'
        disp('Pause movement');
    otherwise, error('Unknown command');
end
```

Task 1: test the mass-range boundary

Change mass_g to 120 g and run the whole section. Identify the first matching mass-category branch and check whether the boundary is included. Update the result comment beside the category display to match the observed output.

Task 2: switch to the Pause command

Restart from the original baseline. Change only the value of cmd to the character vector Pause. Run the section and observe how lower determines the switch case and which movement message is displayed.

3.3 Loops

3.3.1 for, Preallocation and Bounded while

File: `RBE201TC_20261013_3_3.m`

Baseline example: run the whole section first.

```MATLAB
t_mm = [1 3 5];
mass_g = zeros(size(t_mm));
for k = 1:numel(t_mm)
    mass_g(k) = 36*t_mm(k);
end
mass_g   % [36 108 180]
x_m=0; step_m=0.25; target_m=1;
maxSteps=20; count=0;
while x_m<target_m && count<maxSteps
    x_m = x_m + step_m;
    count = count + 1;
end
[x_m count]         % [1 4]
reached = x_m>=target_m; % true
disp(reached);
```

Task 1: increase the while step

Change step_m to 0.5 m while keeping the starting position and 1 m target unchanged. Run the section and inspect the final x_m, count and reached values. Update the result comment beside the position-and-count display to match the observed output.

Task 2: stop at the iteration limit

Restart from the original baseline. Change maxSteps to 2 while retaining the original 0.25 m step and 1 m target. Run the section and inspect the final x_m, count and reached values. Update the two result comments to match the output and distinguish stopping at the iteration limit from reaching the target.

3.3.2 continue, break and Nested Loops

File: `RBE201TC_20261013_3_3.m`

Baseline example: run the whole section first.

```MATLAB
values=[2 NaN 4 10 6]; total=0;
for k=1:numel(values)
    if isnan(values(k))
        continue
    end
    if values(k)>8
        break
    end
    total=total+values(k);
end
A=[1 2;3 4]; B=[5 6;7 8];
assert(isequal(size(A),size(B)));
C=zeros(size(A));
for r=1:size(A,1)
    for c=1:size(A,2)
        C(r,c)=A(r,c)+B(r,c);
    end
end
C   % [6 8;10 12]
disp(total);
disp(k);
```

Task 1: replace break with continue

Replace break with continue only in the branch for a value above 8; retain the continue in the NaN branch. Run the whole section and inspect total and the final k to determine whether the value 6 after 10 is processed.

Task 2: change elementwise addition to subtraction

Restart from the original baseline. Change the nested-loop calculation so each entry of C is the corresponding entry of A minus the corresponding entry of B. Run the section and inspect C and its dimensions, then update its result comment to match the observed matrix.

3.4 Functions

3.4.1 Function Inputs, Outputs and Local Values

File: `RBE201TC_20261013_3_4.m`

Baseline example: run the whole section first.

```MATLAB
[good_C,idx]=filterTemps([35.2 36.5 NaN 37.8 38.1 36.0]);
disp(good_C); disp(idx);
disp(size(good_C)); disp(size(idx));
```

Task 1: call with a column vector

Change only the input in the filterTemps caller to a column vector containing 36, NaN and 38 degrees Celsius, in that order. Keep the local function definition unchanged. Run the section and inspect the values, original indices and dimensions of good_C and idx.

Task 2: valid input with no matches

Restart from the original baseline. Change only the input in the filterTemps caller to a row vector containing 35 and 39 degrees Celsius. Keep the local function definition unchanged. Run the section and inspect good_C and idx to distinguish valid input with no matching values from invalid empty input.

3.4.2 Input Checks and Early Return

File: `RBE201TC_20261013_3_4.m`

Baseline example: run the whole section first.

```MATLAB
tfClose = isEqualTol([1 2],[1 2+1e-10],1e-8);
tfShape = isEqualTol([1 2],[1;2],1e-8);
tfNaN = isEqualTol([1 NaN],[1 NaN],1e-8);
tfInf = isEqualTol([1 Inf],[1 Inf],1e-8);
tfAtTol = isEqualTol(0,0.125,0.125);
tfEmpty = isEqualTol([],[],1e-8);
disp([tfClose tfShape tfNaN tfInf tfAtTol tfEmpty]);
```

Task 1: make both shapes agree

Change only the inputs in the tfShape caller so both are identical column vectors containing 1 followed by 2. Retain the original tolerance and the local isEqualTol definition. Run the section and compare tfShape and the summary display with the original row-versus-column case.

Task 2: increase the strict tolerance

Restart from the original baseline. Change only the tolerance in the tfAtTol caller to 0.25, retaining the compared values 0 and 0.125 and the function's strict less-than rule. Run the section and compare tfAtTol and the summary display with the original tolerance-boundary case.

3.4 Function support: keep at the end of the same file.

These complete functions are already included in the code package. Edit only the callers above.

MATLAB

```
function [T_clean,idx] = filterTemps(T_C)
    bad = isempty(T_C) || ~isnumeric(T_C);
    bad = bad || ~isreal(T_C) || ~isvector(T_C);
    if bad
        error('A nonempty real numeric vector is required.');
    end
    keep = ~isnan(T_C) & (T_C>=36) & (T_C<=38);
    T_clean = T_C(keep);
    idx = find(keep);
end

function tf=isEqualTol(a,b,tol)
if ~isfloat(a) || ~isfloat(b) || ...
        ~isreal(a) || ~isreal(b)
    error('Real floating arrays required.');
end
if ~(isnumeric(tol) && isscalar(tol) && ...
        isreal(tol) && isfinite(tol) && tol>0)
    error('Finite positive tolerance required.');
end
tf=false;
if ~isequal(size(a),size(b))
    return
end
if any(~isfinite(a(:))) || ...
        any(~isfinite(b(:)))
    return
end
tf=all(abs(double(a(:))- ...
          double(b(:))) < tol);
end
```

Extension

Floating-Point Values and Tolerances

File: **RBE201TC_20261013_Extension.m**

Baseline example: run the whole section first.

```MATLAB
x = 0.1+0.2; y = 0.3;
x==y                % false
abs(x-y)            % about 5.55e-17
abs(x-y)<1e-12      % true
format long g       % display only
absTol=1e-10; relTol=1e-8;
tf = abs(x-y) <= absTol + ...
    relTol*max(abs(x),abs(y)); % true
disp(tf);
fprintf('x=%.17g, y=%.17g\n',x,y);
```

Task 1: tighten the absolute tolerance

Start from the unmodified baseline. In one line, tighten the strict absolute-tolerance threshold from 1e-12 to 1e-18 and store and display that decision as strict_tf. Rerun the whole baseline and compare strict_tf with the original combined decision tf; keep the combined rule unchanged.

Task 2: change the display format

Start from the unmodified baseline. Use short numeric display instead of long general display, and append one line displaying x and y together. Change only these two lines. Rerun and compare the final display with the existing 17-significant-digit fprintf output and the equality and tolerance decisions.

Numerical Debugging

File: `RBE201TC_20261013_Extension.m`

Baseline example: run the whole section first.

```MATLAB
total=0;
for k=2:2:20
   total=total+k;
end
disp(total)         % 110: sum of even values
wrong=0;
for k=1:2:20
   wrong=wrong+k;
end
disp(wrong)         % 100: sum of odd values
```

Task 1: shorten the even-loop range

Start from the unmodified baseline. Change the first loop's upper bound from 20 to 10, keeping its start at 2 and step at 2. Rerun and observe total and wrong, then update the first

total-display comment to match the result. Change only these two lines and keep the second loop unchanged.

Task 2: change the step to four

Start from the unmodified baseline. Change the first loop's step from 2 to 4, keeping its start at 2 and upper bound at 20. Rerun and observe the values accumulated and total, then update its display comment to describe the selected even values. Change only these two lines; keep the second loop unchanged.

Statistics and Missing-Value Rules

File: `RBE201TC_20261013_Extension.m`

Baseline example: run the whole section first.

```MATLAB
scores=[50 60 80 90];
mean(scores)        % 70
min(scores)         % 50
max(scores)         % 90
std(scores)         % about 18.2574
std(scores,1)       % about 15.8114
sum(scores>=60)     % 3
sum(scores>mean(scores)) % 2
raw=[50 NaN 80 Inf];
valid=isfinite(raw); % [1 0 1 0]
clean=raw(valid);    % [50 80]
mean(clean)         % 65
std(clean)          % about 21.2132
sum(scores>=90)     % 1
disp(valid); disp(clean); disp(numel(clean));
```

Task 1: replace Inf with a finite value

Start from the unmodified baseline. Replace raw's Inf entry with 20 and append one display line showing clean's valid count, mean and default sample standard deviation, in that order. Change only these two lines. Rerun and inspect the finite-value mask, retained values and summary; keep scores unchanged.

Task 2: use N normalisation

Start from the unmodified baseline. Change the two default standard-deviation evaluation lines to use N normalisation with flag 1. Store and display the scores result as sigma_N and the clean result as sigma_clean_N. Change only these two lines, rerun and compare them with the original sample-deviation values; keep the data and finite-value policy unchanged.

Maximum Values and Their Positions

File: RBE201TC_20261013_Extension.m

Baseline example: run the whole section first.

```MATLAB
M=[1 9;9 5];
[largest,k]=max(M(:));  % largest=9, k=2
[r,c]=ind2sub(size(M),k);
[largest r c]          % [9 2 1]
allK=find(M==largest); % [2;3]
[allR,allC]=ind2sub(size(M),allK);
[allR allC]            % [2 1;1 2]
disp(M(:)); disp([largest k r c]); disp(allK);
```

Task 1: add one tied maximum

Start from the unmodified baseline. In the matrix definition, change only the bottom-right entry from 5 to 9. Edit one line, rerun and compare the first maximum's linear and row-column positions with all tied locations.

Task 2: display all position mappings together

Start from the unmodified baseline. Append one display line combining allK, allR and allC into three columns ordered as linear index, row and column. Rerun and inspect how every tied maximum maps from its linear index to its row-column location; keep the original matrix.

Comparative Plot Panels

File: RBE201TC_20261013_Extension.m

Baseline example: run the whole section first.

```MATLAB
t_mm=1:0.5:5;
m1=36*t_mm; m2=54*t_mm;
figure; subplot(1,2,1);
plot(t_mm,m1,'-o'); hold on;
plot(t_mm,m2,'-s'); hold off;
xlabel('Thickness (mm)');
ylabel('Predicted mass (g)');
legend('200 mm','300 mm');
axis([1 5 0 300]); grid on;
subplot(1,2,2);
plot(t_mm,m2-m1,'-o');
xlabel('Thickness (mm)');
ylabel('Mass increase (g)');
grid on;
% At t_mm=3:
% m1=108 g, m2=162 g
% difference=54 g
difference_g=m2-m1;
disp([t_mm(:) m1(:) m2(:) difference_g(:)]);
```

Task 1: compare a 250 mm plate

Start from the unmodified baseline. Change m2's mass coefficient from 54 to 45 g/mm to represent a 250 mm plate, and change the second legend label from 300 mm to 250 mm. Edit only those two lines. Keep the 200 mm plate, density, width and solid-plate model unchanged; rerun and inspect both mass curves and their difference at 3 mm thickness.

Task 2: narrow the left panel's display

Start from the unmodified baseline. Edit the existing axis-limit line so the left panel shows thickness from 2 to 4 mm and predicted mass from 0 to 150 g. Change one line and rerun. Inspect the markers and line segments at the display boundaries, then compare the stored arrays and numerical table with the right-hand difference panel.

3-D Points, Arrows and Surfaces (Optional)

File: `RBE201TC_20261013_Extension.m`

Baseline example: run the whole section first.

```MATLAB
p=[1;2;3]; figure;
plot3(p(1),p(2),p(3),'o');
hold on;
quiver3(0,0,0,1,2,3,0);
xlabel('x (m)'); ylabel('y (m)');
zlabel('z (m)');
axis equal; grid on; view(3);
[X,Y]=meshgrid(-1:1,-1:1);
Z=X.^2+Y.^2;
figure; subplot(1,2,1);
mesh(X,Y,Z);
subplot(1,2,2);
surf(X,Y,Z);
disp(size(Z)); disp(Z);
```

Task 1: refine the mathematical grid

Optional. Start from the unmodified baseline. In the meshgrid definition, change both grid steps from 1 to 0.5 while retaining the domain from -1 to 1. Edit one line and rerun. Inspect the size, values and minimum and maximum of Z, and compare mesh with surf; retain the same mathematical formula, point and arrow.

Task 2: move the point and arrow together

Optional. Start from the unmodified baseline. Move p to the coordinates (2, 1, 3) m and change the arrow's displacement components to the same values. Edit only the point-definition and quiver3 lines. Keep the arrow's origin at (0, 0, 0), automatic scaling disabled and the mathematical surface unchanged; rerun and inspect the point and arrow together.